0b58b572-64fa-41b0-ab57-b1a4b644a7ad

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'dotnet/aspnetcore' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/aspnetcore/aspnetcore (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-19T13:24:11.328+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Replace Blazor WebAssembly DevServer with Gateway in templates
</pr_title>

<pr_description>
Summary

This PR replaces the Blazor WebAssembly DevServer with the Blazor Gateway in the standalone WASM template, and leverages the SDK's built-in `StaticWebAssetSpaFallbackEnabled` property instead of the custom `SpaFallback.targets`.

Changes

Commit 1: Remove ThreadingApp and multi-threading testing

```
support
Multi-threading is not currently supported. Removes
ThreadingApp, ThreadingApp.Server, associated test files, and
the multi-threading helper infrastructure.

### Commit 2: Use SDK StaticWebAssetSpaFallbackEnabled instead
of custom SpaFallback.targets
- Deletes `src/Components/WebAssembly/testassets/
SpaFallback.targets`
- Adds `StaticWebAssetSpaFallbackEnabled=true` to StandaloneApp
and the benchmark TestApp
- The SDK now generates `{**fallback:nonfile}` endpoints in the
manifest at build time

### Commit 3: Remove MapFallbackToFile from BlazorGateway
Since the SPA fallback is now handled via the static web assets
endpoints manifest (served by `MapStaticAssets`), the Gateway no
longer needs to call `MapFallbackToFile`.

### Commit 4: Update WASM template to use Blazor Gateway instead
of DevServer
- Replaces
`Microsoft.AspNetCore.Components.WebAssembly.DevServer` with
`Microsoft.AspNetCore.Components.Gateway`
- Adds `OverrideHtmlAssetPlaceholders` and
`StaticWebAssetSpaFallbackEnabled` properties

## Dependencies

Requires SDK fix: https://github.com/dotnet/sdk/pull/54353

## Testing

- Built repo successfully
- Verified StandaloneApp E2E with Gateway + SPA fallback
- Verified instantiated blazorwasm template E2E with patched SDK

</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
```

Pull request overview

Removes the multithreading-focused ThreadingApp/ ThreadingApp.Server test apps and supporting infrastructure, deletes the workaround `SpaFallback.targets` in favor of the SDK's `StaticWebAssetSpaFallbackEnabled`, switches the standalone WASM project template from `Microsoft.AspNetCore.Components.WebAssembly.DevServer` to `Microsoft.AspNetCore.Components.Gateway`, and drops the no-longer-needed `MapFallbackToFile` from the Gateway.

Changes:
- Delete ThreadingApp/ThreadingApp.Server, related E2E tests, COOP-header helpers, and `TestTrimmedOrMultithreadingApps` (renamed to `TestTrimmedApps`).
- Replace the custom `SpaFallback.targets` import with the SDK property `StaticWebAssetSpaFallbackEnabled=true` in StandaloneApp, Wasm.Performance.TestApp, GlobalizationWasmApp, and the WASM template; remove `MapFallbackToFile` from `BlazorGateway`.
- Update the WASM template to reference `Microsoft.AspNetCore.Components.Gateway` instead of DevServer, threading the Gateway package version through test infrastructure.

Reviewed changes

Copilot reviewed 57 out of 59 changed files in this pull request and generated 2 comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/ProjectTemplates/TestInfrastructure/ Directory.Build.targets.in:16-21</file>
<original_commit>c18d7c3</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Now that the WASM template no longer references `Microsoft.AspNetCore.Components.WebAssembly.DevServer`, this pinning entry (and the corresponding `MicrosoftAspNetCoreComponentsWebAssemblyDevServerVersion` template variable passed in PrepareForTest.targets, plus the `PackageVersionVariableReference` for DevServer in `Microsoft.DotNet.Web.ProjectTemplates.csproj`) no longer applies to any generated template. The `PackageReference Update` is effectively dead code (no matching PackageReference). Consider removing the DevServer references and only keeping the Gateway pin.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/Components/test/E2ETest/Infrastructure/ServerFixtures/TrimmingServerFixture.cs:15-18</file>
<original_commit>c18d7c3</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Adding `new` here hides the identically-named and identically-initialized field on the base class `BasicTestAppServerSiteFixture<TStartup>` (also updated in this PR). Both fields will now exist on the derived type with the same value, which is wasteful and likely to confuse readers. Consider removing this declaration entirely and inheriting `TestTrimmedApps` from the base class instead.
</comment_old>

<comment_new>
<comment_id>3260807106</comment_id>
<author>@pavelsavara</author>
This sounds legit @javiercn
</comment_new>

<comment_new>
<comment_id>3260807106</comment_id>
<author>@javiercn</author>
@copilot review and fix it
</comment_new>

</comment_thread>

<comment_thread_resolved>
<file>src/Components/test/E2ETest/
Microsoft.AspNetCore.Components.E2ETests.csproj:119</file>
<original_commit>163231e</original_commit>

<comment_old>
<author>@pavelsavara</author>
This needs matching change on many other places I think.

src\Components\test\E2ETest\Infrastructure\ServerFixtures\Trimmi
ngServerFixture.cs
src\Components\test\E2ETest\Infrastructure\ServerFixtures\Blazor
WasmTestAppFixture.cs
src\Components\test\E2ETest\Infrastructure\ServerFixtures\BasicT
estAppServerSiteFixture.cs
</comment_old>

</comment_thread_resolved>

</pr_reviews>

</comments>

The last **4** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement><NoWarn>$
(NoWarn);NU1901;NU1902;NU1903;NU1904</NoWarn> (covers restore
and build), not WarningsNotAsErrors.
- Citations: eng/tools/RepoTasks/RepoTasks.csproj:15-18; src/
Components/Testing/tasks/
Microsoft.AspNetCore.Components.Testing.Tasks.csproj:10-11

testing practices
- Fact: Components/Web.JS has Playwright integration tests
runnable via `npm run test:integration`, using a fixture server
on port 5588 (baseURL http://localhost:5588).
- Citations: src/Components/Web.JS/package.json:11-21; src/
Components/Web.JS/playwright.config.ts:6-19; src/Components/
Web.JS/test/Validation/serve-fixtures.mjs:12-15,97-99

Be sure to consider these stored facts carefully. Consider
whether any are relevant to your current task. If they are,
verify their current applicability before using them to inform
your work.

If you come across a memory that you're able to verify and that

```
you find useful, you should use the store_memory tool to store
the same fact again. Only recent memories are retained, so
storing the fact again will cause it to be retained longer.

If you come across a fact that's incorrect or outdated, you
should use the store_memory tool to store a new fact that
reflects the current reality.
Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address
them. Ignore comments that are not actionable or directed at
you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




<system_reminder>
You have the capability to call multiple tools in a single
response.
For maximum efficiency, whenever you need to perform multiple
independent operations, ALWAYS call tools simultaneously
whenever the actions can be done in parallel rather than
sequentially (e.g. multiple reads/edits to different files).
Especially when exploring repository, searching, reading files,
viewing directories, validating changes. For example, you can
read 3 different files in parallel, or edit different files in
parallel. However, if some tool calls depend on previous calls
to inform dependent values like the parameters, do NOT call
these tools in parallel and instead call them sequentially (e.g.
reading shell output from a previous command should be
sequential as it requires the sessionID).
</system_reminder>
```